

# Manual

By

Leo Steliaros ([las1g24@soton.ac.uk](mailto:las1g24@soton.ac.uk))

Earl Colaço ([erc1g24@soton.ac.uk](mailto:erc1g24@soton.ac.uk))

Sam Patel ([sp3g24@soton.ac.uk](mailto:sp3g24@soton.ac.uk))

## Language Syntax

### Line ends

All variable assignment lines and lines within a WHERE clause must end with a '.' character. Other lines such as lines that end with () or {} have implicit line end characters.

Required line ends:

```
?g1 = "input.ttl" . // variable assignment
{ ?p = ^<http://example/> . } // condition
```

Implicit line ends:

```
SELECT (?s, ?p, ?o) FROM (?g) WHERE {}
```

### Variables

All variables start with the '?' character. Variables can be assigned to values using the '=' character outside of the Querying section, used as references to graphs/triple variables (subject predicate object) within the core structure of SELECT statement or used in conditions to filter/construct triples.

Inside of a querying section the '=' character is instead used to check variable values against a provided value.

Value assignment:

```
?g1 = "input.ttl" . // variable ?g assigned to the string "input.ttl" (located outside of the Query section)
```

Graph value assignment:

```
SELECT (?s, ?p, ?o) FROM (?g) // ?s, ?p, ?o are the subject, predicate and object variables; ?g is a reference to a normalised graph
```

Query condition:

```
{ ?p = ^<http://example/> . } // condition statement, accept only triples where the predicate is a URI equal to http://example/ .
```

Variables within SELECT statements can also be explicitly associated with a graph variable with one of the two following syntaxes:

?graph\_var.?var  
?var in ?graph\_var

## Operators

Operators are used in two main locations, variable assignment before the Query and within WHERE clauses inside Queries. The only operator which can be used outside of a Query is '=' which is used to assign a variable to a value (in this context). Within WHERE clauses, the following operators are valid:

- **MAX** ?var - outputs the triple with the maximum value in the specified variable (per group)
- **MIN** ?var - outputs the triple with the minimum value in the specified variable (per group)
- **COUNT** ?var - outputs triples, replacing the specified variable with the length of the group
- **SUM** ?var - outputs triples, replacing the specified variable with the sum of values of the variable across the group
- **AVG** ?var - outputs triples, replacing the specified variable with the average of values of the variable across the group
- Condition && Condition - logical AND on the result of two conditions
- Condition || Condition - logical OR on the result of two conditions
- ! Condition - logic negation on the result of a condition
- ?var <= variable or rdf term - inclusive less than comparison on ?var
- ?var >= variable or rdf term - inclusive greater than comparison on ?var
- ?var = variable or rdf term - equality comparison on ?var
- ?var < variable or rdf term - less than comparison on ?var
- ?var > variable or rdf term - greater than comparison on ?var

## Graph normalisation

Graphs must be normalised using the 'init' command to be filtered against by queries such as:

?g = init ?g1 ?g2 . // ?g2 is the filepath where the normalised graph is stored.

This is explained in more detail in [Built-In Commands Normalisation](#).

## Graph Querying

Graphs are queried using SELECT statements which can then be combined using UNION, INTER or DIFF statements.

Such graphs must have been [normalised](#) explicitly before usage.

SELECT statements return a list of Triples produced by filtering the Triples of all graphs in the FROM clause against all conditions in the WHERE clause (which may be empty).

SELECT statements have one of the following structures:

1. SELECT (a, b, c) FROM (?g1 , ... , ?gN) // no TO or WHERE clause
2. SELECT (a, b, c) FROM (?g1 , ... , ?gN) TO (?out) // no WHERE clause
3. SELECT (a, b, c) FROM (?g1 , ... , ?gN) WHERE {conditions} // no TO clause
4. SELECT (a, b, c) FROM (?g1 , ... , ?gN) TO (?out) WHERE {conditions} // all clauses

1. Selects all graph lines and writes them to stdout.
2. Selects all graph lines and writes them to a designated file.
3. Selects graph lines that fulfill all condition lines and writes them to stdout.
4. Selects graph lines that fulfill all condition lines and writes them to a designated file.

a is the subject variable and can either be ?s or ?gi.?s where ?gi is a graph in the FROM clause.

b is the predicate variable and can either be ?p or ?gi.?p where ?gi is a graph in the FROM clause.

c is the object variable and can either be ?o or ?gi.?o where ?gi is a graph in the FROM clause.

In the above clauses, if the variable has no explicit graph reference, it is implicitly assigned to the first graph variable in the FROM clause.

UNION, INTER and DIFF all have the same structure and apply the union, intersection or difference operations in order of the branches such as:

```
(  
  Query  
  UNION  
  Query  
)
```

## Built-In Commands

The language uses a few built-in commands which are used both inside and outside Select statements.

### Normalisation

The 'init' command is called to normalise an input graph into a standard form, which can then be filtered against by the condition evaluation logic of the interpreter. For example:

```
?g1 = "normInput.ttl" .  
?g2 = "normOutput.ttl" .  
?g = init ?g1 ?g2 .
```

Is a block of code that:

1. Defines a variable g1 and assigns a string containing a file path
2. Defines a variable g2 and assigns a string containing a file path
3. Defines a variable g and assigns the result of initialising the file referenced in g1 into the file referenced in g2

The contents of the file referenced in g2 are then read by the interpreter into a String variable and used to construct a tuple in the dataset (the list of graph names and associated triples).

?g is then used as the value of the normalised graph stored as a list of triples.

### Select commands

The SELECT statement is a command that defines the structure of output triples, which graphs triples should be selected from, which file the output should be written to (if any) and which conditions the triples should be filtered against (if any).

When a graph variable (for instance ?gi) is added before a triple variable (?s, ?p, or ?o) this means that only the subject, predicate or object of matching triples from the specified graph should be output. This is used in Join operations, such as the construction of new triples where the object of one matches the subject of another.

### Query statement commands

These commands are the statements UNION, INTER and DIFF which apply union, intersection and difference operations on the results of nested queries respectively.

### Aggregation commands

Aggregation commands are commands which are used within the Where clause of a Select statement and return all unique triples that can be constructed. Aggregation commands group triples based on subject, predicate, or object and then return a resultant/constructed triple for each group.

'SUM' constructs a triple for each group, calculating the sum of the specified Triple variable (subject, predicate or object).

'AVG' constructs a triple for each group, calculating the average (rounded by casting result into an Int) of the specified Triple variable.

'COUNT' constructs a triple for each group, returning the length of each group (count of triples in the group) of the specified Triple variable.

'MAX' filters triples, returning the triple with the highest value in the specified Triple variable for each group.

'MIN' filters triples, returning the triple with the lowest value in the specified Triple variable for each group.

## Language Rationale

The language is designed to function similarly to SPARQL, due to the similar goals of querying RDF files.

URIs have to be preceded by ^ due to issues with tokenisation between inequalities and URIs.

## Queries

The primary command is running queries, which can be chained together by UNION, INTER or DIFF commands, to restrict and expand the queries.

Each query uses the SELECT statement to initialise output variables used in the query; a FROM clause where the input RDF files are specified; an optional TO clause to specify the files the query writes to (writing to console otherwise); and an optional WHERE clause that contains the conditions that restrict the query.

The TO clause was initially added for testing purposes, to allow outputs to be written to a file where they could be observed clearly. It also allows for the chaining of queries, as later queries could use the results of previous queries as their inputs provided their input files have the same file paths as the previous query's output file.

## Graph Initialisation

The language also uses an explicit normalise function init, for the purpose of user control over what files would be normalised, however a query is required to be executed on a normalised file. This means that the normaliser is not repeated for the same input file as well, ensuring computational time is not wasted on repeating processes, with the caveat of a more complex initialisation process for the programmer. This trade-off was preferred however, due to the heaviness of the normalisation function in comparison to the rest of the interpreter.

## Graph referencing

Graphs can be either implicitly or explicitly referenced within any variable of a SELECT statement. Graphs are implicitly referenced by default as this reduces the length of a program a programmer needs to write, as well as cleaning up the appearance of programs (especially those where there is only one graph being selected from).

Explicit graph references were added to the language to ensure that programs could distinguish between variables in graphs if the programmer passed more than one graph in the FROM clause.

## Testing Process

The tokenisation process was tested by running the lexer on the input syntax file first and returning the list of tokens to the console to check that the correct tokens are lexed without errors. Then the list of tokens is parsed into an Abstract Syntax Tree, and that tree is then output to the console, to be verified that the order and grouping of all tokens is correct.

The normaliser is tested using an internal function `AutoNormal`, which automatically uses a dedicated testing turtle file, and overwrites a dedicated output turtle file with the normalised turtle file. This allowed for rapid testing without having to deal with nuisances such as mistypes in the file paths.

Additionally, the normaliser was tested directly using GHCi, as its processes could be run independently of the whole stack framework, and GHCi allowed for much more direct testing without having to rely on the rest of the interpreter.

The querying logic was tested in two main stages: the development of the Query module, and then the development of the Main module where changes needed to be made in the Query module.

During the development of the Query module, a main method was defined in the module and the three main methods of the module (`parseRDF`, `unparseRDF`, `executeQuery`) were individually tested using predefined values.

- `parseRDF` was tested on predefined strings in the normalised format.
- `unparseRDF` was tested on predefined lists of Triples, the result of a `parseRDF` call on a predefined string, or the result of an `executeQuery` call.
- `executeQuery` was tested on a predefined AST and predefined dataset (constructed of a graph name and list of Triples) and on a predefined AST and a dataset constructed from the result of calling `parseRDF` on a predefined string in the normalised format

During the development of the Main module, changes had to be made to the Query module to ensure its functionality within the interpreter. The main testing carried out on Query in this stage of development was the creation of actual programs in the language which were lexed and parsed and then used in calls to the `executeQuery` method within the Main module.

The final testing involved creating example program files to test the interpreter and adapting the input turtle files in every possible combination to check for any unintended behaviour, of which none was spotted.

## Language References

SPARQL 1.1 Query Language:

<http://www.w3.org/TR/2013/REC-sparql11-query-20130321/>

## Extras

### File Output

#### Normaliser

During the process of normalisation, the contents of the normalised version of the input RDF graph is written to a file path specified by the user. The contents of this file are later read by the interpreter into a String var called 'rdfdata' which is used to construct the list of triples (in the interpreters custom Triple type) and then an element in the dataset (a list of [graph variable, list of triples]).

This output file is produced when a program contains the 'init' command, which takes two graph variables and writes the normalised version of the first file into the second file.

#### Main

SELECT statements may contain an optional TO clause which contains one or more variables which reference a file path stored as a LitStr. After the interpreter has produced a string to be output (by running executeQuery against the queries and constructed dataset), it produces a tuple containing two lists: 'fromVars' and 'toVars'.

'toVars' is a Maybe [String].

When determining whether to produce a console output or write to a file, the interpreter first checks if the value of 'toVars' is Nothing, outputting the result string to stdout if it is.

Otherwise, an 'outVar' is created, checking the case of 'toVars' again and extracting the first element of the [String] component of 'toVars'. 'outVar' is then resolved using lookup within the environment (list of variable names and associated values) and the result string is written to the file path referenced in 'outVar'. In all other cases, the interpreter informs the user over stdout that it failed to write to the specified file path and is instead outputting the result to console, which is then written on the next line.

### VSCoDe Extension

Extension Name: plc-rdf-interpreter

Most Recent Version: 0.0.6

Example Picture:

```

?g1 = "src/normTest/normInput.ttl" .
?g2 = "src/normTest/normOutput.ttl" .
?g = init ?g1 ?g2 .

(
SELECT (?s ?p ?obj) FROM (?g) WHERE {
    ?p = ^<http://example.org/ont/hasAge> .
    ( ?obj < 21 ) || ( ?obj > 24 ) .
}
UNION
SELECT (?s ?p ?obj) FROM (?g) TO (?out) WHERE {
    ?p = ^<http://example.org/ont/hasAge> .
    ( ?obj > 21 ) || ( ?obj < 24 ) .
}
)

SELECT (?s ?p ?obj) FROM (?g1, ?g2) TO (?out) WHERE {
    ?p in ?g1 = ^<http://example.org/ont/hasAge> .
    MAX ?obj .
}

```

The extension colours URIs and strings blue, numbers as green, top level query statements as purple, inbuilt functions as yellow and variables as orange.

It uses regex to match strings to specific tokens, which are then assigned visual studio code designations that contain implicit colour patterns.

It requires the vsix file to be downloaded and then installed onto visual studio code, after that the extension requires files to be labelled as .rq1 files to prettyprint.

The most recent extension version is saved in the interpreter as  
vscode-extension/plc-rdf-interpreter-0.0.6.vsix